Kiln DeFi Integrations Security Review

Auditors

Optimum, Lead Security Researcher

D-Nice, Lead Security Researcher

Xiaoming90, Security Researcher

0x4non, Associate Security Researcher

Report prepared by: Lucas Goiriz

April 15, 2024

Contents

1 About Spearbit

Spearbit is a decentralized network of expert security engineers offering reviews and other security related services to Web3 projects with the goal of creating a stronger ecosystem. Our network has experience on every part of the blockchain technology stack, including but not limited to protocol design, smart contracts and the Solidity compiler. Spearbit brings in untapped security talent by enabling expert freelance auditors seeking flexibility to work on interesting projects together.

Learn more about us at spearbit.com

2 Introduction

Kiln is a staking platform you can use to stake directly, or whitelabel staking into your product. It enables users to stake crypto assets, manually or programmatically, while maintaining custody of your funds in your existing solution, such Fireblocks, Copper, or Ledger.

Disclaimer: This security review does not guarantee against a hack. It is a snapshot in time of defi-integrations according to the specific commit. Any modifications to the code will require a new security review.

3 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

3.1 Impact

- High - leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium - global losses <10% or losses to only a subset of users, but still unacceptable.
- Low - losses will be annoying but bearable--applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

3.2 Likelihood

- High - almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium - only conditionally possible or incentivized, but still relatively likely
- Low - requires stars to align, or little-to-no incentive

3.3 Action required for severity levels

- Critical - Must fix as soon as possible (if already deployed)
- High - Must fix (before deployment if not already deployed)
- Medium - Should fix
- Low - Could fix

4 Executive Summary

Over the course of 10 days in total, Kiln engaged with Spearbit to review the defi-integrations protocol. In this period of time a total of **31** issues were found.

Summary

Project Name	Kiln
Repository	defi-integrations
Commit	816b6e...bb4c93a
Type of Project	Enterprise Staking, DeFi
Audit Timeline	Jan 1 to Jan 11
Two week fix period	Jan 11 - Jan 28

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	8	7	1
Low Risk	8	7	1
Gas Optimizations	4	2	2
Informational	10	4	6
Total	**31**	**21**	**10**

5 Findings

5.1 High Risk

5.1.1 Invalid asset logic allows for siphoning of performance fee or other user's deposits

Severity: High Risk

Context: Vault.sol#L653-L688

Description: For `_totalAssetsForIncome` and `_totalAssetsForOutcome`, in the event that `currentTotalAssets` is exceeded by `totalPerformanceFees`, they would respectively return `totalPerformanceFees` and `currentTotalAssets`.

The former opens up a scenario where the current vault share holders would siphon a possibly significant portion of the funds deposited from the next depositor to get `currentTotalAssets` to exceed the `totalPerformanceFees`. A proof-of-concept will be provided showcasing a depositor getting 5800× above their fair share of underlying tokens from this, with the depositor losing over 30% from this scenario.

In the latter case, it allows current holders to withdraw, and siphon `totalAssets()` allocated to `collectablePerformanceFees()`, essentially stealing from `feeRecipients` who will be left unable to collect fees. The current holders could be able to withdraw, once more, magnitudes more than they legitimately owned during the positive `currentTotalAssets` balance. This means depositors may be enticed to manipulate `totalAssets()` via flash loans, for underlying assets where this may be possible, to force this condition, although it could also happen naturally from normal market conditions.

Proof of concept:

- Siphoning future depositor:

```
  // 0 offset, max fees
  function test_8923siphon_analyze_more() public {
      uint amount = 100 ether;
      _singleDeposit(alice, amount);
      println("====Alice Initial Deposit====");
      println("alice vault shares = {u}", abi.encode(vault.balanceOf(alice)));
      println("alc withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(alice))));
      println("total assets       = {u}", abi.encode(vault.totalAssets()));

      address(protocol).setTokenBalance(address(asset), asset.balanceOf(address(protocol)) + 50
↪  ether); // emulate 50% TA increase
      println("====Perf Gain====");
      println("alice vault shares = {u}", abi.encode(vault.balanceOf(alice)));
      println("alc withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(alice))));
      println("total assets       = {u}", abi.encode(vault.totalAssets()));

      // alice withdraws majority of her balance + legitimate profits
      _singleRedeem(alice, vault.balanceOf(alice) - 0.001 ether);
      println("====Alice First Withdraw====");
      println("alice vault shares = {u}", abi.encode(vault.balanceOf(alice)));
      println("alc withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(alice))));
      println("total assets       = {u}", abi.encode(vault.totalAssets()));

      println("====Loss Calc====");
      // total asset loss needed to reach performance downturn for stealing the performance fees
      int neededLoss = (int(vault.collectablePerformanceFees()) - int(vault.totalAssets()) - 1);
      println("neededLoss         = {i}", abi.encode(neededLoss)); // with this scenario, this
↪  resolves to 0.01% loss of totalAssets, to trigger siphoning

      // total asset loss occurs
      address(protocol).setTokenBalance(address(asset),
↪  uint(int(asset.balanceOf(address(protocol))) + neededLoss));
      println("====Bob Deposits after perf loss====");
```

```
        uint bobSent = vault.previewMint(vault.balanceOf(alice) * 1);
        _singleDeposit(bob, bobSent);
        println("alice vault shares = {u}", abi.encode(vault.balanceOf(alice)));
        println("alc withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(alice))));
        println("bob vault shares   = {u}", abi.encode(vault.balanceOf(bob)));
        println("bob withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(bob))));
        println("bob TOK sent       = {u}", abi.encode(bobSent)); // Bob loses over 67% of the value
↪   sent here (35% is warranted from fee, rest is from invalid logic to Alice)
        println("total assets       = {u}", abi.encode(vault.totalAssets()));

        println("====Alice Redeem====");
        _singleRedeem(alice, vault.balanceOf(alice)); // alice under this scenario withdraws over
↪   5800x of their legitimate balance
        println("total assets       = {u}", abi.encode(vault.totalAssets()));

    }
```

Output:

```
  [PASS] test_8923siphon_analyze_more() (gas: 1266472)
  Logs:
    ====Alice Initial Deposit====
    alice vault shares = 65000000000000000000
    alc withdraw amt   = 65000000000000000000
    total assets       = 650000000000000000000
    ====Perf Gain====
    alice vault shares = 65000000000000000000
    alc withdraw amt   = 97499999999999999999
    total assets       = 1150000000000000000000
    ====Alice First Withdraw====
    alice vault shares = 1000000000000000
    alc withdraw amt   = 1500000000000000
    total assets       = 17501500000000000001
    ====Loss Calc====
    neededLoss         = -1500000000000002
    ====Bob Deposits after perf loss====
    alice vault shares = 1000000000000000
    alc withdraw amt   = 8749999999999986876
    bob vault shares   = 1000000000000000
    bob withdraw amt   = 8749999999999986876
    bob TOK sent       = 26923076923076896157
    total assets       = 34999999999999982502
    ====Alice Redeem====
    total assets       = 26249999999999995626
```

- Siphoning the performance fee out

```
    // 0 offset, max fees
    function test_siphon_perf_fee() public {
        uint amount = 100 ether;
        _singleDeposit(alice, amount);
        println("====Alice Initial Deposit====");
        println("alice vault shares = {u}", abi.encode(vault.balanceOf(alice)));
        println("alc withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(alice))));
        println("total assets       = {u}", abi.encode(vault.totalAssets()));

        address(protocol).setTokenBalance(address(asset), asset.balanceOf(address(protocol)) + 50
↪   ether); // emulate 50% TA increase
        println("====Perf Gain====");
        println("alice vault shares = {u}", abi.encode(vault.balanceOf(alice)));
        println("alc withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(alice))));
        println("total assets       = {u}", abi.encode(vault.totalAssets()));
```

```
        // alice withdraws majority of her balance legitimate profits
        _singleRedeem(alice, vault.balanceOf(alice) - 0.001 ether);
        println("====Alice First Withdraw====");
        println("alice vault shares = {u}", abi.encode(vault.balanceOf(alice)));
        println("alc withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(alice))));
        println("total assets       = {u}", abi.encode(vault.totalAssets()));

        println("====Loss Calc====");
        // total asset loss needed to reach performance downturn for stealing the performance fees
        int neededLoss = (int(vault.collectablePerformanceFees()) - int(vault.totalAssets()) - 1);
        println("neededLoss         = {i}", abi.encode(neededLoss)); // with this scenario, this
↪   resolves to 0.01% loss of totalAssets, to siphon fees
        println("vCPF               = {i}", abi.encode(vault.collectablePerformanceFees()));

        // total asset loss occurs
        address(protocol).setTokenBalance(address(asset),
↪   uint(int(asset.balanceOf(address(protocol))) + neededLoss));
        println("====Perf Loss====");
        println("alice vault shares = {u}", abi.encode(vault.balanceOf(alice)));
        println("alc withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(alice))));
        println("total assets       = {u}", abi.encode(vault.totalAssets()));
        println("====Alice Redeem====");
        _singleRedeem(alice, vault.balanceOf(alice));
        println("total assets       = {u}", abi.encode(vault.totalAssets()));
        println("====Attempt Perf Fee Collect====");
        _collectPerformanceFees(vault);

    }
```

Output:

```
[FAIL. Reason: ERC20InsufficientBalance(0xDa66f7547eeae6c40575433f575987B995C0b018, 17499
↪  [1.749e4], 17500000000000000000 [1.75e19])] test_siphon_perf_fee() (gas: 1356958)
Logs:
  ====Alice Initial Deposit====
  alice vault shares = 65000000000000000000
  alc withdraw amt   = 65000000000000000000
  total assets       = 65000000000000000000
  ====Perf Gain====
  alice vault shares = 65000000000000000000
  alc withdraw amt   = 97499999999999999999
  total assets       = 115000000000000000000
  ====Alice First Withdraw====
  alice vault shares = 1000000000000000
  alc withdraw amt   = 1500000000000000
  total assets       = 17501500000000000001
  ====Loss Calc====
  neededLoss         = -1500000000000002
  vCPF               = 17500000000000000000
  ====Perf Loss====
  alice vault shares = 1000000000000000
  alc withdraw amt   = 17499999999999982500
  total assets       = 17499999999999999999
  ====Alice Redeem====
  total assets       = 17499
  ====Attempt Perf Fee Collect====

Test result: FAILED. 0 passed; 1 failed; 0 skipped; finished in 11.48ms

Ran 1 test suites: 0 tests passed, 1 failed, 0 skipped (1 total tests)

Failing tests:
Encountered 1 failing test in test/Vault.t3_fees.sol:Vault3Test
[FAIL. Reason: ERC20InsufficientBalance(0xDa66f7547eeae6c40575433f575987B995C0b018, 17499
↪  [1.749e4], 17500000000000000000 [1.75e19])] test_siphon_perf_fee() (gas: 1356958)
```

Recommendation: Under these conditions, instead of returning `totalPerformanceFees` or `currentTotalAssets`, it seems more correct to return `0`, as we're just trying to avoid an underflow from the subtraction operation.

```
  function _totalAssetsForIncome(uint256 totalPerformanceFees, uint256 currentTotalAssets)
      internal
      pure
      returns (uint256)
  {
      return
-         totalPerformanceFees > currentTotalAssets ? totalPerformanceFees : currentTotalAssets -
↪  totalPerformanceFees;
+         totalPerformanceFees > currentTotalAssets ? 0 : currentTotalAssets - totalPerformanceFees;
  }
```

And the likewise action for `_totalAssetsForOutcome`, setting the ternary true-clause, `currentTotalAssets` to 0. From internal testing, the proof-of-concepts provided are no longer an issue with this fix and behaved more expectantly.

Kiln: Fixed in commit e2f8f479.

Spearbit: The prior affected logic has been replaced and is no longer affected by the outlined attacks.

5.2 Medium Risk

5.2.1 Lack of partial performance fee collection could lock uncollected amounts in vault

Severity: Medium Risk

Context: Vault.sol#L736-L750

Description: The `collectPerformanceFees()` function optimistically attempts to distribute the entire `VaultStorage._collectablePerformanceFees` amount, and doesn't account for cases where the underlying assets of the vault may have reached a condition below the fees. In cases where it cannot withdraw the entire amount, the function will fail and has no possibility for partial collections, leaving them locked and uncollectable until `totalAssets()` exceeds the fee.

Proof of concept:

```
// 0 offset, max fees
function test_2987_collect_fees_fail() public {
    uint amount = 100 ether;
    _singleDeposit(alice, amount);
    println("====Alice Initial Deposit====");
    println("alice vault shares = {u}", abi.encode(vault.balanceOf(alice)));
    println("alc withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(alice))));
    println("total assets       = {u}", abi.encode(vault.totalAssets()));

    address(protocol).setTokenBalance(address(asset), asset.balanceOf(address(protocol)) + 50 ether);
↪   // emulate 50% TA increase
    println("====Perf Gain====");
    println("alice vault shares = {u}", abi.encode(vault.balanceOf(alice)));
    println("alc withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(alice))));
    println("total assets       = {u}", abi.encode(vault.totalAssets()));

    // alice withdraws fully
    _singleRedeem(alice, vault.balanceOf(alice));
    println("====Alice Full Withdraw====");
    println("alice vault shares = {u}", abi.encode(vault.balanceOf(alice)));
    println("alc withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(alice))));
    println("vCPF               = {u}", abi.encode(vault.collectablePerformanceFees()));
    println("total assets       = {u}", abi.encode(vault.totalAssets()));

    println("====Loss Calc====");
    // total asset loss needed to trigger insufficient assets for perf fee collection
    int neededLoss = (int(vault.collectablePerformanceFees()) - int(vault.totalAssets()) - 1);
    println("neededLoss         = {i}", abi.encode(neededLoss));

    // total asset loss occurs
    address(protocol).setTokenBalance(address(asset), uint(int(asset.balanceOf(address(protocol))) +
↪   neededLoss));
    println("====Perf Loss====");
    println("vCPF               = {u}", abi.encode(vault.collectablePerformanceFees()));
    println("total assets       = {u}", abi.encode(vault.totalAssets()));
    println("====Attempt Perf Fee Collect====");
    _collectPerformanceFees(vault);
}
```

Output:

```
[FAIL. Reason: ERC20InsufficientBalance(0xDa66f7547eeae6c40575433f575987B995C0b018,
↪  17499999999999999999 [1.749e19], 17500000000000000000 [1.75e19])] test_2987_collect_fees_fail()
↪  (gas: 1221922)
Logs:
  ====Alice Initial Deposit====
  alice vault shares = 65000000000000000000
  alc withdraw amt   = 65000000000000000000
  total assets       = 650000000000000000000
  ====Perf Gain====
  alice vault shares = 65000000000000000000
  alc withdraw amt   = 97499999999999999999
  total assets       = 1150000000000000000000
  ====Alice Full Withdraw====
  alice vault shares = 0
  alc withdraw amt   = 0
  vCPF               = 175000000000000000000
  total assets       = 175000000000000000001
  ====Loss Calc====
  neededLoss         = -2
  ====Perf Loss====
  vCPF               = 175000000000000000000
  total assets       = 174999999999999999999
  ====Attempt Perf Fee Collect====

Test result: FAILED. 0 passed; 1 failed; 0 skipped; finished in 9.45ms
```

Recommendation: If management fees and performance fees will generally be at parity, and have the same breakdown of recipients, consider instead directly setting aside vault tokens during minting, which would act as both, and having the fee manager, appropriately redistribute them among fee recipients. This would remove the necessity for much of the current fee logic, and also make them direct stakeholders, and allow the fee recipients to withdraw freely at their own accord, while also having both virtually charged fees being part of the protocol.

If the current fee breakdowns are required, consider implementing an additional function with partial collection logic, which should generally also be beneficial to the vault even in scenarios outside this, where fee recipients only wish to take a partial amount out of the vault and thereby increasing potential protocol earnings for the vault.

Kiln: Fixed in commit e2f8f479.

Spearbit: Fixed: fees are accounted as shares at their accrual points and should no longer reach the condition where they can exceed actual total assets.

5.2.2 Assets and shares calculations lose user value in vault and can still be frontrun for griefing attacks

Severity: Medium Risk

Context: Vault.sol#L611-L626

Description: The calculations utilized in `_convertToShares` and `_convertToAssets` implement a `_decimalsOffset` variable intended to introduce "*virtual assets*" with the main goal being to restrict traditional ERC4626 share price manipulation attacks from being profitable for an attacker. Even with `_decimalsOffset` set to 0, it mitigates the attack from being directly profitable, in terms of an attacker being unable to siphon the value of other subsequently depositing users to their own share(s).

However, this implementation does cause a value loss to all vault holders across the board, when it comes to their initial deposits and for subsequent gains/losses, essentially lowering the capital efficiency of the vault. The original implementing team considered this to be negligible, which it could be under ideal scenarios, but this mitigation is intended for a non-ideal attack scenario.

Zero (value utilized in tests) or low `_decimalsOffset` values are still vulnerable to a degree of griefing attacks akin to the original share price manipulation attack, the addition of the `PreviewZero()` check helps lower potential losses. An attacker is still able to frontrun the contract in such a way that the socialized losses of the attack among

legitimate users exceeds their own cost of the attack. These attacks can also be performed in such ways that the aforementioned losses inherent to this calculation mechanism are amplified to nontrivial amounts for all users, and also forecfully control the minimum deposit amount and thereby DoS legitimate user's deposits.

Proof of concept: This proof of concept is pluggable into `Vault.t.sol`. It assumes no fees for simplicity and consistency, management fee would just amplify perceived loss, and performance fee would increase attackers cost, as shown in the derived formula:

```
fullSetup(true, 0 * 1e18, 0 * 1e18, managementFees, performanceFees);
```

```
function test_0_offset_grief_attack() public {
    uint bobsAmount = 100 ether;
    // artificial increase of totalAsset balance prior to deposits results in inflation and losses
↪   starting with up to 33%
    AssetMock(address(asset)).mint(address(alice), 1000 ether);
    _singleDeposit(alice, 1);
    println("alice vault shares = {u}", abi.encode(vault.balanceOf(alice)));
    alice.impersonateOnce();
    asset.transfer(address(protocol), bobsAmount); // this formula is ((first user expected amount) *
↪   (1 - mgmt fee)) / (1 - perf fee), we frontrun bob

    _singleDeposit(bob, bobsAmount);
    uint256 toWithdraw = vault.previewRedeem(vault.balanceOf(bob));
    // bob already lost 33% of his deposit, even with no fees
    println("bob vault shares   = {u}", abi.encode(vault.balanceOf(bob)));
    println("bob withdraw amt   = {u}", abi.encode(toWithdraw));
    uint256 toWithdrawA = vault.previewRedeem(vault.balanceOf(alice));
    println("alc withdraw amt   = {u}", abi.encode(toWithdrawA));
    // 33% of total asset value is stuck in contract at this point, any future depositors will
↪   socialize losses
    println("total assets       = {u}", abi.encode(vault.totalAssets()));
    // single future depositor would cause losses among legit depositors to exceed attackers lost
↪   value/cost
}
```

Output:

```
[PASS] test_0_offset_grief_attack() (gas: 542612)
Logs:
  alice vault shares = 1
  bob vault shares   = 1
  bob withdraw amt   = 6666666666666666667
  alc withdraw amt   = 6666666666666666667
  total assets       = 20000000000000000001
```

Recommendation: Scaling `_decimalsOffset` to a sufficiently high value with respect to the underlying asset it represents, makes an attack harder to pull off and should minimize the socializing of losses across legitimate users a griefer could pull off, and instead the griefer would carry a brunt of the losses. There would still be a precision loss, but it should converge to more neglible levels. Another drawback of this, is that it makes the vault tokens a non-default decimal amount which could create user and UX friction and incompatibilities. In addition, utilizing the offset introduces other issues such as valueless fractional shares, regardless of fraction being 1/100 or 99/100.

Another solution which is simpler, and should similarly mitigate inflation attacks and additionally the noted griefing attacks, without introducing any user value loss as a consequence of the calculations is to simply require a minimum `totalSupply()` threshold to be met, for deposits/withdrawals to succeed or any supply changing functionality. This attains an effect similar to the 'virtual shares' mitigation, and higher `totalSupply()` thresholds essentialy increase the upfront economic cost to achieve price manipulation. In addition to this `_decimalsOffset` would need to removed along with `_convertToShares` and `_convertToAssets` refactored.

Examples of the suggested changes for this solution:

```
  function _convertToShares(uint256 assets, Math.Rounding rounding, uint256 total) internal view
↪  returns (uint256) {
-       return assets.mulDiv(totalSupply() + 10 ** _decimalsOffset(), total + 1, rounding);
+       return assets.mulDiv(totalSupply() > 0 ? totalSupply() : 1, total > 0 ? total : 1, rounding);
  }

  function _convertToAssets(uint256 shares, Math.Rounding rounding, uint256 total) internal view
↪  returns (uint256) {
-       return shares.mulDiv(total + 1, totalSupply() + 10 ** _decimalsOffset(), rounding);
+       return shares.mulDiv(total > 0 ? total : 1, totalSupply() > 0 ? totalSupply() : 1, rounding);
  }
```

and at any points of potential `totalSupply` changes such as `deposit`:

```
  function _deposit(
      // ...
      ) internal {
          uint256 _balanceBefore = IERC20(asset()).balanceOf(address(this));
          SafeERC20.safeTransferFrom(IERC20(asset()), caller, address(this), assets);
          _mint(receiver, shares);
+         if(totalSupply() < minSupplyState) revert MinSupplyUnmet();

          VaultStorage storage $ = _getVaultStorage();
```

An additional novel approach for the secondary recommendation, to help ensure the shares are 'dead' and don't end up being burnt, while avoiding any users shares ever becoming 'dead', is to create a purpose built contract that will simply hold the minimum supply for vaults or minting the minimum on behalf of an unownable/dead address such as one of the precompiles, `address(1)`.

With either recommendation, appropriate test scenarios should be included in the coverage, along with general scenarios that don't include any fees and ensure that under no scenario a user loses more than 1 wei of value, when depositing and then withdrawing, with no value changing actions in between. For the secondary solution without inherent precision loss, there will not to be updates to existing test cases which attempt to account for precision loss within their assertions such as Vault.t.sol#L374-L375 which acccounts for `totalAssets` being 1, when it should be 0 following the withdrawal of all user amounts + fees.

Kiln: Ongoing fix. We will implement the minimum `totalSupply()`.

Spearbit: Fix partially implemented. Griefing attacks that can cause non-trivial losses among other users, should be minimized when using sane minimum supply totals such as `1e18`, regardless of decimals offset set in the vault. The team will continue to utilize asset calculations with decimals offset being accounted for, which is a further security guarantee, but does result in accruing precision loss over time, which was a compromise the OpenZeppelin library team that implemented it considered acceptable for the security added.

5.2.3 Assets could be transferred out of sanctioned accounts

Severity: Medium Risk

Context: Vault.sol#L713

Description: One could circumvent the sanction and transfer assets out of sanctioned accounts. Consider the following scenario:

1. Bob grants Alice max allowance to spend his vault shares for certain reasons.
2. Bob gets sanctioned. When one gets sanctioned, their accounts should be frozen, and no movement of assets or vault shares should occur.
3. The `notSanctioned` modifer only blocks Bob from accessing the vault functions. However, it does not prevent Alice from accessing Bob's shares within his account.

4. Alice calls `transferFrom` to pull vault shares from Bob's sanctioned account to her account and redeem them on behalf of Bob.
5. Alice transfers the redeemed assets back to Bob. Bob effectively circumvented the sanction.

Recommendation: The `transferFrom` and `transfer` functions should be overwritten to check that the `to` and `from` addresses are not in the sanction list.

A sanctioned account should not have outgoing or incoming assets because the account is considered frozen.

KILN: Fixed in 875b8d01

Spearbit: Fixed in 875b8d01 by implementing the auditor's recommendation.

5.2.4 Non-compliance to ERC-4626

Severity: Medium Risk

Context: Vault.sol

Description: Several instances of non-compliance to ERC-4626 standards were observed:

- **Instance 1**: `maxDeposit` does not consider the market's supply cap. The ERC-4626 requirements for the `maxDeposit` function are as follows:

  > Maximum amount of the underlying asset that can be deposited into the Vault for the receiver, through a deposit call.
  >
  > MUST return the maximum amount of assets deposit would allow to be deposited for receiver and not cause a revert, which MUST NOT be higher than the actual maximum that would be accepted (it should underestimate if necessary).

  Aave V3 introduces the supply cap parameter, controlled by Aave Governance. If a reserve has a supply cap, this limits the total amount of the asset that can be supplied.

  Assume that the current supply cap is 1 million USDC, and the current total supplied is 950,000 USDC. This means that only an additional 50,000 USDC can be supplied to the pool. Otherwise, the transaction will revert.

  Thus, the maxDeposit function should be overwritten in the respective connector contracts to return 50,000 USDC instead of `type(uint256).max/(1 * 10 ** IERC20Metadata(asset()).decimals())` USDC in this scenario.

  This issue applied to all external markets that Kiln integrated with (e.g., Compound, Morpho) that has a supply cap.

- **Instance 2**: `maxMint` does not consider the market's supply cap. The ERC-4626 requirements for the `maxMint` function are as follows:

  > Maximum amount of shares that can be minted from the Vault for the receiver, through a mint call.
  >
  > MUST return the maximum amount of shares mint would allow to be deposited to receiver and not cause a revert, which MUST NOT be higher than the actual maximum that would be accepted (it should underestimate if necessary).

  Similar to the issue highlighted in Instance 1, the supply cap of the external markets must be considered.

- **Instance 3**: Vault's functions do not consider pause event. The ERC-4626 requirements state that the `maxDeposit`, `maxMint`, `maxWithdraw`, and `maxRedeem` functions must return zero if these functions are disabled.

  > MUST factor in both global and user-specific limits, like if [function] are entirely disabled (even temporarily) it MUST return 0.

  As such, when these functions or the connectors that these functions rely on are paused, they must return zero.

Recommendation: Ensure that the functions of the vault comply with the ERC-4626.

Kiln: Addressed in commit 4278de7e.

Spearbit: Partially Fixed. Instance 3 has been remediated in commit 4278de7e while the fixes for Instance 1 and 2 should be updated to be more closely aligned to the AAVE's supply cap logic (ValidationLogic.sol#L81-L88).

5.2.5 Unable to claim rewards from Compound V3

Severity: Medium Risk

Context: CompoundV3Connector.sol#L52

Description: Per Compound V3 docs, there are potential rewards for accounts that use the protocol. However, the connector does not provide a way to claim the rewards.

Recommendation: Consider implementing the logic required to claim the rewards from Compound V3 in the connector.

Kiln: Fixed in 875b8d01.

Spearbit: Fixed in 875b8d01 by implementing the auditor's recommendation.

5.2.6 Calls to Compound's v2 `cToken` contracts may silently fail, potentially causing loss of funds

Severity: Medium Risk

Context: CompoundV2Connector.sol#L41, VenusConnector.sol#L27

Description: In early versions of the Compound protocol, functions returned error codes instead of reverting in unwanted scenarios. Interacting with these contracts without handling potential error codes might be disastrous to the caller contract. In the current version of the code, `CompoundV2Connector` does not take into account `cToken` contracts that are still based on error codes. One example for such contract might be cUSDC which is still actively used, another set of contracts will be the Venus `vToken` contracts, and vUSDC is one concrete example.

In the context of this system, a call to `CompoundV2Connector.deposit/withdraw` that will silently fail means that the deposited USDC will be forever locked in the cUSDC contract. Since the time is limited for this audit, we will try to cover more areas of the repository before trying to show a specific silent failure scenario as we think it is still crucial to apply the recommendation below even if currently we could not find any specific attack vector.

Recommendation:

- **Short term:** Consider changing the way Compound's contracts (`cToken`, `comptroller`, `comp`) are called in `CompoundV2Connector`. Return values should be carefully checked and the entire transaction should be reverted in case of a non zero error code. Please make sure to verify that return values of current/future versions of Compound's contracts still return the same values in the same order.
- **Long term:** Consider adding sanity checks for `IConnector.deposit`,`IConnector.withdraw` that will wrap the calls to these functions and will make sure that the `IConnector.totalAssets` is increasing/decreasing as expected. Here is a code snippet for inspiration for the `Vault._deposit` function (should be tested), a similar logic should be applied for `_withdraw` as well:

```
function _deposit(
    address caller,
    address receiver,
    uint256 assets,
    uint256 shares,
    uint256 managementFeeAmount,
    uint256 performanceFeeAmount
) internal {
    uint256 _balanceBefore = IERC20(asset()).balanceOf(address(this));
    SafeERC20.safeTransferFrom(IERC20(asset()), caller, address(this), assets);
    _mint(receiver, shares);

    VaultStorage storage $ = _getVaultStorage();
    uint256 amountToDeposit = IERC20(asset()).balanceOf(address(this)) - _balanceBefore -
↪  managementFeeAmount;
    // Deposit to underlying protocol
    address _connector = $._connectorRegistry.getOrRevert($._connectorName);

    // Query the total assets before deposit
     uint256 totalAssetsBefore = totalAssets();
    _connector.functionDelegateCall(
        abi.encodeCall(
            IConnector.deposit,
            (IERC20(asset()), amountToDeposit)
        )
    );
    // Query the total assets after deposit
     uint256 totalAssetsAfter = totalAssets();
    // The diff should be greater than equal what was deposited
    require(totalAssetsAfter - totalAssetsBefore >= amountToDeposit, "Insufficient totalAssets");
    $._collectablePerformanceFees += performanceFeeAmount;
    $._lastTotalAssets = totalAssetsAfter;

    emit Deposit(caller, receiver, assets, shares);
}
```

Regardless, consider having an internal policy for testing(unit tests, integration tests with mocks, integration tests with "forking", testnet deployment, etc) interactions of new connectors as well as making sure your dev team have a firm understanding of how each third party protocol works before deploying new connectors.

Kiln: Ongoing fix. Even if we can solve this issue with a router checking the `totalAssets()` increase or decrease (long term), we will check the returned value (short term).

Spearbit: Partially fixed since return values are not checked in the `claim` function flow in both mentioned contracts and since the invariant checks for deposit and withdraw are not implemented.

5.2.7 Reinvestment of performance fees is not handled correctly in the system

Severity: Medium Risk

Context: Vault.sol

Description: Performance fees are defined as fees taken from profits. Unlike many financial institutions in the real world with a clear interval of when performance fees are supposed to be taken (quarterly, bi-annually, annually, etc), in the current system performance fees are being taken continuously instead - i.e. state changing functions in the system calculate the fee based on the last checkpoint.

These fees are denominated and accounted in base tokens (USDC for example), but are being left to gain yield in the external defi protocol used in the vault's connector. Performance fees can be withdrawn from the underlying defi protocol in the form of base tokens upon calling `collectPerformanceFees` which can later be claimed by fee recipients when `dispatchFees` is being called.

The following describes an issue with how the system handles performance fees:

Yield earned on accumulated performance fees is allocated only for vault share holders and not for fee recipients; while it might sound like a legitimate design decision to make, it might be unwanted in certain scenarios. Let's consider the following short example:

- Alice is the first depositor, she deposits 1000 USDC to the AAVE aUSDC vault and gets 1000 shares (assuming no management fees).
- After a while, the vault aUSDC balance is 1100. At this point Alice is calling `redeem` to burn all of her shares, assuming 20% performance fee, Alice will get 1080 USDC while 20 aUSDC will be left to earn yield in the vault.
- Time passes and now the vault aUSDC balance is 25, it is important to mention that there are no depositors that should benefit from the yield. At this point, the first depositor to call `deposit` will add `20% * (25 - 20) = 1` to `$._collectablePerformanceFees` but the rest 4 aUSDC tokens will be effectively added to his balance since he will hold the shares.

To summarize, depositors might earn yield even though they did not have "*skin in the game*" for that period.

Recommendation: We recommend on one of the following mitigations:

- Consider not re-investing performance fees in the vault at all and convert to the base token every time they are accumulated.
- Consider accounting the performance fees as vault shares (instead of `$._collectablePerformanceFees`) that will be minted to the vault contract address and will be burned upon a call to `collectPerformanceFees`. Keep in mind that management and performance fees should not be taken from these amounts.
- Consider isolating the checkpointed performance fees from the vault contract by transferring the fees (as aUSDC) to a different contract that only the `FeeDispatcher` contract can withdraw from upon `_dispatchFees`.

Kiln: Fixed in commit e2f8f479.

Spearbit: Fixed in e2f8f479 by implementing the second alternative offered by the auditors.

5.2.8 Wrong rounding of division may lead to a denial of service in a specific scenario also invalidating `maxWithdraw` in `Vault._previewWithdraw`

Severity: Medium Risk

Context: Vault.sol#L595-L609

Description: Users can acquire shares in a vault by either calling `Vault.deposit` or `Vault.mint`. To burn the shares and get the base token, they can use either `Vault.withdraw` or `Vault.redeem`. Both functions will transfer the base token back to the user, but in the first one the user can specify the amount of base token to withdraw where in the latter he should specify the amount of equivalent shares.

During the conversion between shares and assets, the rounding policy should be consistent, which is not the case in the case of `Vault.withdraw`. `Vault.withdraw` is rounding up (ceiling value) the result of division (more precisely - the amount of `shares` to send), which can cause the transaction to revert in certain scenarios, where the alternative will be to being able to withdraw but not the full amount, thus losing some tokens.

To better understand the issue let's consider the following example:

1. Alice deposits 1000 usdc to the vault with the connector of AAVE aUSDC vault. Assuming that before her deposit `totalSupply = 0`, management fee is 2%, performance fee is 20%:
   - Alice will hold 980 shares, 20 usdc will be allocated as management fee.

$$lastTotalAssets = 980 \times 10^6$$

Now, let's assume that the vault earned 120 USDC, after applying the performance fee, Alice will be able to withdraw $980 + (1100 - 980) \times 0.8 = 1076$ USDC.

2. Let's assume the dApp UI is calling `Vault.maxWithdraw(Alice)` which will return 1076 USDC and now Alice calls `Vault.withdraw(assets=1076*10^6)`:

$$shares = \left\lceil \frac{(1076 \times 10^6) \times (980 \times 10^6 + 1)}{(1076 \times 10^6 + 1)} \right\rceil = 980,000,001$$

But since Alice only has 980000000 shares, the call to `_burn(Alice, 980000001);` later in the execution will revert which will cause the entire transaction to revert and Alice won't be able to withdraw the amount of tokens entitled to her fully. Although in reality, if Alice will try to withdraw one USDC less (1075 USDC), she will be able to successfully do it. However, users are not supposed to be familiar with the implementation details and for her it might be perceived as that the entire amount is locked. Either way, she will lose 1 USDC for rounding up instead of down.

It is important to mention that as long as Alice's position has positive yield, and as long as she is willing to claim the entire assets amount then `shares = 980000001` (pretty easy to prove mathematically, we don't think that it is necessary in this case).

As we can see since the transaction will fail, it means that the result returned by `maxWithdraw` is inaccurate, thus invalidating the correctness of this function as well.

Recommendation: A workaround will be to make sure that the dApp UI implements the flow of full withdrawal by calling `redeem` instead of `withdraw`.

Kiln: Acknowledged. When converting, the rounding is here to protect the protocol and avoid a loss on the protocol side if the rounding is not favorable.

Spearbit: Acknowledged.

5.3 Low Risk

5.3.1 Partial valueless shares may be minted or transfered/sold when `_decimalsOffset() > 0`

Severity: Low Risk

Context: Vault.sol#L20, Vault.sol#L623-L626

Description: In the case `_decimalsOffset()` is set to a non-zero value, it allows for partial shares to be minted, and in the case of `transferable` being true, transfer of partial shares. In the case of an offset of 4, shares in the range of 1-999 could be minted and transferred.

In every case, these fractional shares are valueless without withdrawal power, and when it comes to minting is not beneficial to end-users, and results in lost user value. In the transfer case, this could be abused to sell an amount of shares within that range on a secondary market, while being valueless to that buyer within the vault itself, as it would have no withdrawal power. This could lead to delisting of the shares due to this issue.

Proof of concept:

```
// scenario where valueless fractionless shares are sold, possibly exceeding their nominal value
// on a secondary market, but are valueless within this vault itself, the greater value is possible
// as there'd be no management fee on these
// assumes 0% fee, offset 18 and transferable true
function test_8932_valueless_partial_shares_transfer() public {
    uint bobAmount = 1e18;
    _singleDeposit(alice, vault.previewMint(bobAmount));
    println("====Alice Initial Deposit====");
    println("alice vault shares = {u}", abi.encode(vault.balanceOf(alice)));
    println("alc withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(alice))));
    println("total assets       = {u}", abi.encode(vault.totalAssets()));

    alice.impersonateOnce();
    vault.transfer(address(bob), 1e18 - 1);
    println("====Alice Post Transfer====");
    println("alice vault shares = {u}", abi.encode(vault.balanceOf(alice)));
    println("alc withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(alice))));
    println("total assets       = {u}", abi.encode(vault.totalAssets()));

    _singleMint(bob, bobAmount);
    println("====bob Initial Deposit====");
    println("ETH bob sent       = {u}", abi.encode(vault.previewMint(bobAmount)));
    println("bob vault shares   = {u}", abi.encode(vault.balanceOf(bob)));
    println("bob withdraw amt   = {u}", abi.encode(vault.previewRedeem(vault.balanceOf(bob))));
    println("total assets       = {u}", abi.encode(vault.totalAssets()));
    _singleWithdraw(bob, vault.previewRedeem(vault.balanceOf(bob)));
    println("Bob withdrew");
    println("total assets       = {u}", abi.encode(vault.totalAssets()));
}
```

Output:

```
[PASS] test_8932_valueless_partial_shares_transfer() (gas: 835921)
Logs:
  ====Alice Initial Deposit====
  alice vault shares = 1000000000000000000
  alc withdraw amt   = 1
  total assets       = 1
  ====Alice Post Transfer====
  alice vault shares = 1
  alc withdraw amt   = 0
  total assets       = 1
  ====bob Initial Deposit====
  ETH bob sent       = 1
  bob vault shares   = 1999999999999999999
  bob withdraw amt   = 1
  total assets       = 2
  Bob withdrew
  total assets       = 1
```

Recommendation: Ensure the remainder of minted and transferred shares is 0 when doing modulo of the amount against `10 ** _decimalsOffset()` during either action. This should ideally be within the base ERC4626 implementation itself. This attack is likely not economically worth unless a base unit of the underlying asset, or secondary market value, at least exceeds the transactional gas cost of the network.

Kiln: Ongoing fix.

Spearbit: Fixed. Would also recommend having test cases ensuring partial shares are rejected when offset is set.

5.3.2 Incorrectly using underlying decimals for `maxMint`

Severity: Low Risk

Context: Vault.sol#L316-L319

Description: The `maxMint` function should relay to users the maximum vault shares that may be minted. The function currently returns the maximum possible that could fit into a 256-bit uint type, divided by the decimal unit of the underlying asset, which in most cases will be 10^{18}.

This assumes a constant synchronization between the decimal units of the underlying asset and vault shares. There are a number of cases where this assumption may fail.

1. If using a non-standard token decimals contract, and there is a failure in acquiring decimals during initialization by the vault, it will fallback to 18 as seen here ERC4626Upgradeable.sol#L99. In this case the synchronization assumption would not hold.
2. If utilizing `_decimalsOffset` as a OpenZeppelin ERC4626 likely should to protect against a class of attacks, it would cause a disparity between the underlying decimals and the actual minted amounts, as `_decimalsOffset` is not accounted for. This could cause an early bricking of `mint` via `ERC4626ExceededMaxMint` error. Therefore, shares you may be able to get with a deposit, you would not be able to get with mint anymore.

Recommendation: This issue is being posted to note these potential faulty scenarios with the current implementation, their current logic is not conformant to the ERC4626 standard as specified by the issue "Non-compliance to ERC-4626" and the recommendations there supercede this issue and should be implemented while still accounting for the scenarios noted here, and to ultimately avoid a desync between `maxDeposit` and equivalent `maxMint` values.

Kiln: Fixed in commit e15e2fe5.

Spearbit: Fixed.

5.3.3 Missing sanity check on `_setOffset` could result in bricked contract

Severity: Low Risk

Context: Vault.sol#L880-L885, Vault.sol#L611-L626

Description: `_setOffset` sets the `_offset` parameter for `VaultStorage`, which amplifies the supply to protect against a class of attacks. The parameter passed is a `uint8` type, however, sufficiently high values could still be passed in the 8-bit space that could cause an immediate or eventual bricking of the vault from resulting overflows.

An immediate bricking would not result in user losses, as the deposit mechanism's should be bricked, but an eventual bricking would result in user losses, where the parameter passed is not high enough to result in bricking immediately, but eventually once a certain degree of deposits have entered the vault. For example, values exceeding 76 would result in immediate bricking. Slightly lower ranges would result in eventual bricking in accordance with deposits/total asset increases, bricking both deposits and withdrawals.

Recommendation: Introduce some sanity checks within `_setOffset` to ensure its passed offset won't cause an overflow state. The general range of valid input for `decimalsOffset()` discussed by OZ has been the 0-9 range. The actual safe value is dependent on the total supply of the underlying asset, so it may vary, but considering the currently highest circulating supply cryptocurrency, with about 1e18 circulation, and assuming 18 decimals, up to 23 is technically safe, albeit a limit of 19 should offer sufficient protection, minimize rounding errors for 18 decimal underlying assets inherent from the calculation mechanism, and leave sufficient room for any current realistic circulating supplies.

Kiln: On going fix. We will set a max value of 23.

Spearbit: Fixed.

5.3.4 The claim might revert in certain situations where the vault incurs a loss

Severity: Low Risk

Context: Vault.sol#L760

Description:

```
function claimAdditionalRewards(address rewardsAsset, bytes calldata payload)
    external
    nonReentrant
    onlyRole(CLAIM_MANAGER_ROLE)
{
    VaultStorage storage $ = _getVaultStorage();

    address connector = $._connectorRegistry.getOrRevert($._connectorName);
    connector.functionDelegateCall(
        abi.encodeCall(IConnector.claim, (IERC20(asset()), IERC20(rewardsAsset), payload))
    );

    uint256 _totalAssets = totalAssets();
    if ($._lastTotalAssets > _totalAssets) {
        revert TotalAssetsDecreased($._lastTotalAssets, _totalAssets);
    } else if ($._lastTotalAssets == _totalAssets) {
        revert NoAdditionalRewardsClaimed();
    }
```

Assume `$._lastTotalAssets` is 1000 USDC and `_totalAssets` is 950 USDC. So, there is a loss of 50 USDC.

Assume that a certain number of rewards are waiting to be claimed that will allow the vault to gain *x* USDC. If the gain of *x* USDC is equal to or less than 50 USDC, the claim function will always revert.

The claim function should not have reverted because the TX increased the number of assets in the vault.

Recommendation: If the first invariant check intends to ensure that the total assets within the vault do not decrease after the connector's `claim` function is executed, consider comparing the value of `totalAssets()` before and after the connector's `claim` function. Same recommendation applies also for the issue "Protection against draining of external vault tokens is not perfect in `Vault.claimAdditionalRewards`".

Kiln: Fixed in 875b8d01.

Spearbit: Fixed in commit 875b8d01. The total assets held by the vault before and after a claim are checked to ensure that it does not decrease.

5.3.5 Invariant checks fail to detect outgoing assets during claim due to outdated `_lastTotalAssets`

Severity: Low Risk

Context: Vault.sol#L760

Description: The `claimAdditionalRewards` function implements invariant checks to ensure that the vault's total assets must not decrease after the `connector.claim` function is executed. This is to guard against the claim manager from intentionally or accidentally swapping out the assets during the claim process.

After successfully claiming the additional rewards, the total assets held by the vault will increase. However, the `_lastTotalAssets` was not updated to the current total assets at the end of the `claimAdditionalRewards` function. As a result, the following edge-case scenario might happen where the invariant checks fail to detect outgoing assets during the claim.

Assume the following (also assume that there is no user interaction between first and second claims):

1. Current `totalAssets()` = 100 ETH, `$._lastTotalAssets` = 100 ETH.
2. The first claim is executed and increases the `totalAssets()` by 10 ETH. The `totalAssets()` = 110 ETH and `$._lastTotalAssets` remain at 100 ETH.

3. The second claim is executed. However, 1 ETH is intentionally or accidentally swapped out of the vault when the `connector.claim` function is executed. Thus, the `totalAssets()` decreased by 1 ETH to 109 ETH.
4. The first invariant check at Line 774 within the `claimAdditionalRewards` function will pass because `$._lastTotalAssets > _totalAssets` ⇒ `100 ETH > 109 ETH` ⇒ `false` ⇒ No Revert.
5. The second invariant check at Line 775 within the `claimAdditionalRewards` function will pass because `$._lastTotalAssets == _totalAssets` ⇒ `100 ETH == 109 ETH` ⇒ `false` ⇒ No Revert.
6. Both invariant checks fail to detect outgoing assets during the claim.

Recommendation: Consider updating the `_lastTotalAssets` to the latest value whenever the total assets in the vault changes.

Kiln: Fixed in 875b8d01.

Spearbit: Fixed in 875b8d01. The total assets held by the vault before and after a claim are checked to ensure that it does not decrease.

5.3.6 New vaults can be created with the wrong connector registry in `VaultFactory.createVault`

Severity: Low Risk

Context: VaultFactory.sol#L89-L96

Description: During the execution of `VaultFactory.createVault`, the `connectorRegistry_` parameter is being overwritten with the immutable value of `connectorRegistry`. The issue, however, is that this over-writing operation is only made after the old value of `connectorRegistry_` was encoded to `_initCalldata`.

Issue is labeled as low risk since the function can only be called by the `DEPLOYER_ROLE` and it is assumed to be trusted.

Recommendation: Consider changing the order of the two lines so it will be:

```
params.connectorRegistry_ = connectorRegistry; // overwrite the connector registry address
bytes memory _initCalldata = abi.encodeCall(Vault.initialize, params);
```

Kiln: Fixed in 875b8d01.

Spearbit: Fixed in 875b8d01 by implementing the auditor's recommendation.

5.3.7 No conversion of decimals between external third party DeFi tokens and `asset()`

Severity: Low Risk

Context: Vault.sol#L300-L304

Description: The system implemented in this repository acts as a wrapper for users to deposit base tokens (USDC for example) and earn yield on it by depositing this tokens to other DeFi protocols. Many DeFi protocols mint tokens that act as a certificate of deposit of the depositor (aUSDC for example). There are two main ways to represent the certificate of deposit: one as shares in the pool of total deposits and the other as a rebasing token. The issue that we are describing here is more relevant to the latter.

It is important to mention that normally, the decimals of the base token and its corresponding pool/vault token are identical, but there could be edge cases of protocols that don't follow this best practice. It is also important to mention that for the `totalAssets` calculations the system queries the **rebasing** token balance and not the `asset()` (base token) itself.

For instance, in theory, aUSDC could have been implemented with 18 decimals instead and it would be up to the integrator (the Kiln team for instance) to make the conversion. Failing to do so, may result in severe loss of funds although not likely and thus this issue is labeled as low severity.

Recommendation: Consider implementing the conversion between the two different decimals of the different tokens in `Vault.totalAssets()`. If you are choosing not to, at least make the practice of verifying that both

decimals are identical as part of the proposal of new connectors inside the dev team and at least include comments about the research in `IConnector.totalAssets`.

Kiln: After discussing the issue, we prefer to manage this on the connector side if needed.

Spearbit: Acknowledged.

5.3.8 Protection against draining of external vault tokens is not perfect in `Vault.claimAdditionalRewards`

Severity: Low Risk

Context: Vault.sol#L760-L780, CompoundV2Connector.sol#L89-L110

Description: The system implemented in this repository acts as a wrapper for users to deposit base tokens (USDC for example) and earn yield on it by depositing this tokens to other defi protocols. Some of the defi protocols add another incentives in the form of allocating rewards for depositors in other tokens (Compound's COMP token for example), `Vault.claimAdditionalRewards` facilitates the claim of these reward tokens.

During the claim, the accepted reward tokens are converted from the reward tokens to the base token and then deposited into the DeFi protocol. It is important to mention that the function is restricted for `CLAIM_MANAGER_ROLE` only, which is trusted to a certain extent but since we spotted verification checks that make sure this trusted user does not act maliciously we decided to file an issue since these checks are not perfect and may still allow him to steal funds from the system in very specific scenarios, let's look at two potential ones:

1. Assuming that in a future connector, the `rewardsAsset` injected by the `CLAIM_MANAGER_ROLE` in the `Vault.claimAdditionalRewards` is being used (Currently it is not the case and each connector uses its own predetermined reward token luckily). The `CLAIM_MANAGER_ROLE` can decide to inject the vault token (`cToken` for example) and by doing so **potentially** to steal any yields made up until this point (he will need to donate some base tokens to the `Vault` address during the hijacking of the program execution somehow but for the sake of example let's assume it is possible).
2. The `swapTarget` might be a multicall contract or/and have a function to swap multiple tokens. In case `swapTarget` is approved to spend the LP tokens (`cToken` for example) they might be drained, since a malicious caller can swap both the reward tokens but also the LP tokens (only the current yield). another scenario might be that the `swapTarget` contract is a multicall contract and the same contract where the `assset()` is deposited and which allows swapping the rewards tokens. In this case a malicious caller can call functions like withdraw, borrow or other functions that will allow him to move the LP tokens to his account.

To summarize, the described scenarios are very unlikely but might have a high impact, that is why we decided to label this issue as low severity but with a **strong recommendation** to fix these issues.

Recommendation:

1. Consider removing the `rewardsAsset` parameter from `Vault.claimAdditionalRewards` or at least add a "blacklist" of addresses that the provided `rewardsAsset` can not be part of.
2. In both cases, the validation check in the end of the `Vault.claimAdditionalRewards` is not implemented correctly since it allows a potential malicious `CLAIM_MANAGER_ROLE` to steal yield accrued from the last check. Consider changing this check so the diff will be `totalAssetsBeforeTheClaim - totalAssetsAfterTheClaim` instead of `$._lastTotalAssets - totalAssetsAfterTheClaim`.

Kiln Fixed in commit 875b8d01.

Spearbit: Fixed in commit 875b8d01 by implementing the auditor's recommendation.

5.4 Gas Optimization

5.4.1 `for` loop increment gas optimization

Severity: Gas Optimization

Context: FeeDispatcher.sol#L131-L159

Description: Depending on optimization flags and settings in the compiler, there are some optimizations possible with singularly incrementing `for` loops, with greater gas savings seen the more loopings there are.

Recommendation: In the case the contract will be compiled with the optimization flag `off`, there are gas savings to utilizing `++i` over `i++`. Use the former if that is the case, in the case optimization flag is on, there should be no difference with the `0.8.22` Solidity compiler or later.

Additionally, in any optimization flag case, using an unchecked block for the increment would yield further gas savings, generally safe if the loop control variable is `uint256`. This is recommended only for simpler control flows without continues for example.

For the contextualized loop:

```
- for (uint256 i; i < _recipientsLength; i++) {
+ for (uint256 i; i < _recipientsLength;) {
      currentRecipient = $._feeRecipients[i];

      if (_pendingManagementFee > 0) {
          // Compute the management fee amount for the current recipient (based on the management
          // fee split between all recipients).
          uint256 managementFeeAmount = _pendingManagementFee.mulDiv(
              currentRecipient.managementFeeSplit, _MAX_PERCENT * 10 ** underlyingDecimals
          );
          if (managementFeeAmount != 0) {
              asset.safeTransfer(currentRecipient.recipient, managementFeeAmount);
              _managementFeeTransfered += managementFeeAmount;
              emit ManagementFeeDispatched(currentRecipient.recipient, managementFeeAmount);
          }
      }

      if (_pendingPerformanceFee > 0) {
          // Compute the performance fee amount for the current recipient (based on the performance
          // fee split between all recipients).
          uint256 performanceFeeAmount = _pendingPerformanceFee.mulDiv(
              currentRecipient.performanceFeeSplit, _MAX_PERCENT * 10 ** underlyingDecimals
          );
          if (performanceFeeAmount != 0) {
              asset.safeTransfer(currentRecipient.recipient, performanceFeeAmount);
              _performanceFeeTransfered += performanceFeeAmount;
              emit PerformanceFeeDispatched(currentRecipient.recipient, performanceFeeAmount);
          }
      }
+     unchecked { ++i; }
  }
```

Kiln: The `optimizer` will be set at `true` (by default). With the current Solidity version (`0.8.22`), we won't add this optimization..

Spearbit: Acknowledged.

5.4.2 Variable packing opportunities

Severity: Gas Optimization

Context: proxy/VaultUpgradeableBeacon.sol#L43-L50, ConnectorRegistry.sol#L51-L59, Vault.sol#L102-L113

Description: Along the codebase, there are certain places where variables could be packed to save storage slots. In particular:

- VaultUpgradeableBeacon.sol#L43-L50: the `_implementation`, `pauseTimestamp` and `frozen` variables are stored in separate slots. This approach is inefficient because these variables are often read together, and accessing storage coldly is costly in terms of gas.
- A similar issue is present in ConnectorRegistry.sol#L51-L59, where each variable is associated with a separate mapping. This setup is not only inefficient but could also be optimized by packing the variables into a single slot.
- Finally, there is another opportunity in Vault.sol#L102-L113 where this struct can also be packed to avoid extra cold reads.

Recommendation:

- VaultUpgradeableBeacon.sol#L43-L50: pack the `_implementation`, `pauseTimestamp` and `frozen` variables into a single slot. This can be achieved by changing the representation of `pauseTimestamp` from `uint256` to `uint88`. Since these variables are read together, packing them will avoid costly cold reads from storage.
- Similarly, in ConnectorRegistry.sol#L51-L59, the variables can be packed into one mapping, thereby saving on gas and optimizing storage usage.
- The `VaultStorage` struct could also be optimized. All the fee variables could use a lower integer representation since they have a defined maximum value, and could be combined with a reordering of variables to optimize slot usage.

KILN: Fixed in commits 1d9198a1, ca6d8cda, a745c411 and 3eea146b.

Spearbit: Fixed.

5.4.3 Calls to `accrueInterest` are redundant in `CompoundV2Controller` and `VenusConnector`

Severity: Gas Optimization

Context: CompoundV2Connector.sol#L79-L109, VenusConnector.sol#L65-L95

Description: `cToken.accrueInterest` is a function that syncs the recent accumulated interest for the borrow and supply sides. It is being called already as part of `cToken.mint`,`cToken.redeemUnderlying` in both versions of the `cToken` contract (older version can be found in cUSDC and newer in cUSDT), and thus there is no need to call this function before and after the interaction.

Recommendation: Consider removing these calls to improve gas efficiency.

Kiln: Ongoing fix.

Spearbit: Partially fixed in commit 875b8d01 since `cToken.accrueInterest();` was not removed from the `claim` function in both mentioned contracts.

5.4.4 Gas optimizations on `FeeDispatcher`

Severity: Gas Optimization

Context: abstracts/FeeDispatcher.sol

Description/Recommendations:

1. `_dispatchFees`: Consider using operators like `>,<` instead of `!=` since the latter might be compiled to two different opcodes instead of one.
2. `_setFeeRecipients`: Is implementing an array de-duplication which being done in $\mathcal{O}(n^2)$ while it can be optimized by requiring an ordered list (should be strictly ordered by `FeeRecipient.recipient`) and just validated on-chain that the list is indeed strictly sorted (by requiring that `i.recipient < i + 1.recipient` for instance) which will cost only $\mathcal{O}(n)$ where n is the number of recipients.
3. `_dispatchFees`: Currently the transfer of `asset` is made in two different calls, one for `managementFeeAmount` and the other for `performanceFeeAmount`. These two calls are not necessary and instead can be replaced by a single transfer of `managementFeeAmount + performanceFeeAmount`.

Kiln:

1. Ongoing fix.
2. Acknowledged. We do not want to optimize so much for a function that is called so few times.
3. Acknowledged (same as reason as above).

Spearbit: Acknowledged.

5.5 Informational

5.5.1 Add `supportsAsset` function to validate asset compatibility

Severity: Informational

Context: Vault.sol#L857-L862

Description: The `_setConnectorName` function allows setting a new connector name if it exists within the `_connectorRegistry`. However, there's no validation to ensure that the assets managed by the vault are intended for or compatible with the specified connector. This could potentially lead to misconfigurations where vaults are connected to incompatible connectors, leading to operational errors.

Recommendation: Add to the connector's functionality a `supportsAsset(address)` function. This function would take an `asset()` and return a boolean indicating whether the asset is intended for and compatible with the connector. This additional validation step can be integrated into the `_setConnectorName` function or wherever connectors are assigned to vaults, to ensure that only compatible connectors can be assigned to manage specific assets.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.5.2 Use open `pragma` on interfaces

Severity: Informational

Context: src/interfaces

Description: The `pragma` directive in the interfaces is not use an open `pragma`. Utilizing open `pragma`s is considered a good practice for enhancing compatibility and ensuring that the interface can work with a broader range of compiler versions without being too restrictive or too permissive.

Recommendation: Adopt open `pragma` directives for all interfaces. This approach not only aligns with the best practices but also improves the integration with contracts. Since the interfaces do not have error statements, adjusting the `pragma` directive to something like `pragma solidity >=0.7.0 <0.9.0;` could be a balanced approach, ensuring compatibility with a range of compiler versions while avoiding too broad a range that might include versions with breaking changes.

Kiln: Acknowledged, we will keep the fixed pragma.

Spearbit: Acknowledged.

5.5.3 Metamorphic implementation could change logic even if updates frozen

Severity: Informational

Context: VaultUpgradeableBeacon.sol#L175-L179, VaultUpgradeableBeacon.sol#L86-L90, VaultUpgradeableBeacon.sol#L122-L128

Description: *An important preface to this is that metamorphic contracts will become obsolete by the upcoming Dencun update/fork on the Ethereum mainnet, and thus this issue will* ***not*** *apply. EVMs that don't implement a similar update with the associated EIP-6780 changes or similar, will still be potentially vulnerable to this*.

The current design of the vaults is to be upgradeable and allow changing the implementation address. There is additionally functionality to `freeze()` the vaults to the most recent implementation, optically taking away the centralization behind the upgrade mechanism. However, if this most recent implementation is a metamorphic contract, normally requiring deployment via `CREATE2` and some methodology to access a working `SELFDESTRUCT` opcode, the deployer or another authorized party could still potentially change the logic, even if it appears to be optically frozen.

Recommendation: This is informational, as there are no indications within the current implementation logic that will be used as of this commit hash, that `SELFDESTRUCT` is utilized or the implementation being deployed via `CREATE2`. In the case of Ethereum Mainnet, following the above noted update, this will be a non-issue, therefore it wouldn't matter even if the implementation were deployed via `CREATE2`. In the case of other EVMs, it is recommended to continue without deploying the implementations via `CREATE2`.

However, in the case of vulnerable EVM chains, there are tools such as metamorphic-contract-detector to help users and devs detect potentials for this, and the implementation in use, specifically if frozen, should be ensured to not be metamorphic to ensure it is in fact frozen.

Kiln: Acknowledged, we will keep the `VaultFactory` with `CREATE2` on all EVM chains so it's easier to maintain the same version. On the other hand, we must be sure that the `SELFDESTRUCT` opcode is inaccessible.

Spearbit: Acknowledged.

5.5.4 An alternative design could prevent the loss of collected performance fees when `totalAssets() < _collectablePerformanceFees`

Severity: Informational

Context: Vault.sol

Description: The total performance fee accumulated could potentially make up a significant percentage (e.g., 98%) of the total assets in the vault due to one or more following factors:

- The performance fee is not collected for a long period.
- A large number of users withdraw from the vaults, resulting in the number of assets owned by users reducing significantly.
- A significant drop in the market value of the yield tokens held by the vault.

Assume that the vault's total assets (`vault.totalAssets()`) are 1000 USDC. Of this amount, 99 USDC is allocated for collected performance fees (`_collectablePerformanceFees`), and the remaining 901 USDC belong to the users.

Due to a certain market condition, a large number of users redeem their shares from the vault, resulting in the vault's total assets being reduced by 900 USDC to 100 USDC. At this point, the performance fee collected (99 USDC) makes up most of the vault's total assets (100 USDC).

When the vault reaches this state, one could potentially take advantage of how the total assets are computed during deposit and withdrawal to gain a profit within single or multiple blocks if they could accurately anticipate an upcoming drop in the exchange rate (e.g. daily rebase or share price update transaction in mempool). Profiting within a single block requires sandwiching the exchange rate update transaction with the deposit and withdrawal transactions.

The following shows how the total assets are computed during deposit and withdrawal below.

```
function _totalAssetsForIncome(/*...*/) { // called during mint and deposit
    return totalPerformanceFees > currentTotalAssets ? totalPerformanceFees : currentTotalAssets -
↪  totalPerformanceFees;
}

function _totalAssetsForOutcome(/*...*/) { // called during redeem and withdraw
    return totalPerformanceFees > currentTotalAssets ? currentTotalAssets : currentTotalAssets -
↪  totalPerformanceFees;
}
```

Notice that if the conditional statement flips from True To False or vice versa, the Price Per Share (PPS) might change dramatically, which allows one to exploit significant discrepancies in the PPS before and after the "flip" occurs.

In *T*1, the exchange rate is 1.0, and the vault state is as follows:

Performance fee collected	Vault's total assets	Total supply	PPS
99 USDC	100 USDC	10 shares	0.1 USDC

When the `_totalAssetsForIncome` function is executed during deposit, the condition will be false as the performance fee is not larger than the vault's total assets. As such, the total assets used for computing the number of shares minted to the users will equal `currentTotalAssets - totalPerformanceFees` (100 USDC - 99 USDC = 1 USDC). PPS equals 0.1 USDC per share.

Bob deposited 1 USDC and 10 shares are minted to him. After Bob's deposit, the vault state is as follows:

Performance fee collected	Vault's total assets	Total supply	PPS
99 USDC	101 USDC	20 shares	0.1 USDC ((101 - 99 USDC)/ 20 shares

At *T*2, the exchange rate drops slightly from 1.0 to 0.98. As such, the total assets of the vault drop from 101 USDC to 98 USDC.

Bob redeems all his 10 shares. When the `_totalAssetsForOutcome` function is executed, the condition will be true as the performance fee is larger than the vault's total assets. As such, the total assets used for computing the number of shares minted to the users will equal `currentTotalAssets` (98 USDC).

With 20 shares in the vault, the PPS will be 4.9 USDC per share (98 USDC / 20 shares), a sharp increase in PPS (4800% increase). Bob will receive 49 USDC in return, effectively earning a profit of 48 USDC (4800% profit). This occurs because when the performance fee is larger than the vault's total assets, the integrator forfeits all their performance fee collected to avoid an underflow that could block withdrawal.

Usually, the protocol keeps all the fees it collects and does not leak any assets to users under any circumstances. However, the present design represents a departure from this principle.

Recommendation: It was understood that this was a deliberate design choice to use the collected performance fee as the total assets if the `totalAssets() < _collectablePerformanceFees` to avoid underflow and prevent the vault from locking up. When this occurs, the protocol/integrator indirectly forfeits all their collected performance fee to allow the remaining users to withdraw from the vault.

However, this scenario could be avoided if the performance fee has been segregated from the users in the first place. Some possible ways to segregate assets can be found in the "Recommendation" section of issue "Reinvestment of performance fees is not handled correctly in the system".

Kiln: Addressed in commit e2f8f479.

Spearbit: Fixed in e2f8f479.

5.5.5 Unpausing connectors on removal to prevent re-adding in paused state

Severity: Informational

Context: ConnectorRegistry.sol#L186-L189

Description: When a connector is removed, its corresponding entry in the `_connectors` mapping is deleted. However, the `pauseTimestamp` for the removed connector is not cleared. This oversight could lead to a situation where if a connector with the same name is added again in the future, it would inherit the previous connector's `pauseTimestamp`, potentially starting in a paused state if the timestamp is set to a future date.

Recommendation: Delete the connector's entry in the `pauseTimestamp` mapping upon removal. This can be achieved by adding the line `delete pauseTimestamp[name];` in the `remove` function. This change ensures that if a connector is re-added with the same name, it does not start in a paused state due to residual data from the previously removed connector.

Kiln: Fixed in commit d2790656.

Spearbit: Fixed.

5.5.6 Misleading, incorrect, and duplicated NatSpec comments

Severity: Informational

Context: abstracts/FeeDispatcher.sol#L117, ConnectorRegistry.sol#L96, proxy/VaultUpgradeableBeacon.sol#L124, connectors/MorphoCompoundV2Connector.sol#L31

Description: See below a list with all the misleding, wrong or duplciated comments:

- abstracts/FeeDispatcher.sol#L117: Comment is duplicated with `L120`.
- ConnectorRegistry.sol#L96: The comment `Throws if the connector is frozen.` should say `paused` instead of `frozen`.
- VaultUpgradeableBeacon.sol#L124: The comment says that `owner` can upgrade the beacon, when its the role `IMPLEMENTATION_MANAGER_ROLE` who can upgrade the beacon.
- MorphoCompoundV2Connector.sol#L31: The title of the contract makes reference to Compound V3 but it is the Compound V2 connector.

Recommendation:

- Remove duplicate comment on `FeeDispatcher.sol#L117`.
- On `ConnectorRegistry.sol#L96` change comment to `Throws if the connector is paused.`
- On `proxy/VaultUpgradeableBeacon.sol#L124` comment should say `msg.sender must have the ``IMPLEMENTATION_MANAGER_ROLE`` role.`
- On `MorphoCompoundV2Connector.sol#L31` change contract title to `Morpho Compound V2 Connector.`

KILN: Fixed in PR 51.

Spearbit: Fixed.

5.5.7 Inconsistent usage of `asset` in `MetamorphoConnector` and `SDAIConnector`

Severity: Informational

Context: MetamorphoConnector.sol#L40, SDAIConnector.sol#L41

Description: `IConnector.deposit` first parameter is the asset that should be deposited to the third party DeFi protocol and is expected to be deposited to the `Vault` contract before that. In both `MetamorphoConnector` and `SDAIConnector`, the implementation of `deposit` ignores the value of this parameter and instead makes use of immutable variables instead. In the case of an erroneous deployment script, the addresses of the `Vault.asset()` and the addresses used in these connectors might be different which will cause any deposit to fail, and will require privileged users to step in and change the connector of this vault with an upgrade which is not desirable and may impact the reputation of the protocol.

Although this issue is labeled as informational, we **strongly recommend** to fix it to reduce any possible surface area around it.

Recommendation: Consider removing the immutable variables used and instead stick to the original `deposit` interface wherever possible.

Kiln: Fixed in 875b8d01.

Spearbit: Fixed in 875b8d01 by implementing the auditor's recommendation.

5.5.8 `VenusConnector` and `CompoundV2Connector` code is almost identical but still duplicated

Severity: Informational

Context: VenusConnector.sol, CompoundV2Connector.sol

Description: Venus contracts are forked from Compound v2 and as such, their interfaces and functions are almost identical, however, the connectors for these protocols do not share a common code but instead their code is duplicated.

Duplicate code, or code that is copied and pasted multiple times within a project or across projects, is generally not a good practice in software development. It can lead to several issues, including increased maintenance costs, decreased code readability, and a higher likelihood of introducing bugs into the codebase. When code is duplicated, any changes that need to be made must be replicated across all instances of the code, which can be time-consuming and error-prone.

Additionally, duplicated code can make it harder to understand the overall structure of the codebase, as well as make it more difficult to identify and fix issues when they arise. Issues like "Calls to Compound's v2 cToken contracts may silently fail, potentially causing loss of funds" will require fixes in two contracts instead of one in case of sharing the code.

Recommendation: Consider using inheritance to make sure the code for the two contracts is shared instead of having to duplicate it.

Kiln: Acknowledged, we prefer to keep two contracts and consider them as two different protocols. Even if we can use inheritance, it's a personal preference.

Spearbit: Acknowledged.

5.5.9 Events with `Updated` suffix should include the previous value as well

Severity: Informational

Context: Vault.sol#L131, ConnectorRegistry.sol#L73

Description: Events are meant to be a cheap storage alternative to provide a way to audit state changes off-chain. Normally when changing a state variable it is recommended to also include the old value of the variable before the change.

Recommendation: Consider including the old value as well in all events with an `Updated` suffix.

Kiln: Acknowledged, only emitting the new value is good on our end.

Spearbit: Acknowledged.

5.5.10 No explicit slippage checks for swaps of liquidity mining reward tokens

Severity: Informational

Context: Vault.sol#L760-L780

Description: The system implemented in this repository acts as a wrapper for users to deposit base tokens (USDC for example) and earn yield on it by depositing this tokens to other defi protocols. Some of the defi protocols add another incentives in the form of allocating rewards for depositors in other tokens (Compound's COMP token for example), `Vault.claimAdditionalRewards` facilitates the claim of these reward tokens. During the claim, the accepted reward tokens are converted from the reward tokens to the base token and then deposited into the defi protocol. During the conversion, no explicit slippage checks are made on the Vault contract, as these checks suppose to happen as part of the `swapTarget` contract which might not be always the case.

Recommendation: To make the system more robust, consider adding a check inside the `Vault.claimAdditionalRewards` flow to make sure that the minimum required amount is being received. You may want to add a parameter that represents this minimum amount to `claimAdditionalRewards` although it is supposed to be part of the `payload` parameter for convenience.

Kiln: Acknowledged. Since slippage is already accounted for in the payload, we prefer not to add another parameter. This approach maintains flexibility within the payload parameter.

Spearbit: Acknowledged.